CORE ATTENDEE EDITION: Start with 00_START_HERE.md at the pack root. Only the core two-day route is supplied. Optional publishing, dummy website, resource creation, identity/networking and advanced Azure tracker extensions mentioned below are not assigned or included. Use your own extracted path where a command shows C:\YOUR-ATTENDEE-PACK. Do not copy instructor settings.

Pre-course setup and prerequisites

LBS | Windows attendee guide | 28 October 2026 | Core attendee edition

Complete before the two-day in-person course. Both days run 09:45-17:30. Day 1 begins with a 15-minute readiness check, not a software-installation workshop. Allow 60-90 minutes plus any corporate IT approval time before class; contact the organiser early if blocked.

1. Choose your participation route

Full developer route: portal, VS Code, terminal SQL/Blob/KQL, local Function and Day 2 local tracker. Install the route-required tools below before arrival.

Portal-only supported route: browser and approved Azure access are enough for assigned portal SQL, Blob and read-only views. VS Code, Visual Studio, .NET SDK, Git and Core Tools are not mandatory for portal-only participation. Local/terminal exercises remain Not run or tutor-observed unless you later prepare those tools.

Prepared/offline route: when billing or telemetry access is unavailable, the Day 2 fictional data supports reasoning practice. Running prepared KQL still requires an authorised query editor; reading CSVs offline does not. Neither route establishes live billing access or telemetry collection.

Default to existing assigned Azure resources. Creation permissions, resource-provider registration and deployment rights are needed only for a separately assigned creation/publishing exercise. Never request Owner merely to pass setup.

Tool / access	Required for	Not required for
Current supported Edge/Chrome and MFA	All portal activities; approved organisational identity	Offline file review
VS Code	The course's VS Code and terminal route; recommended common editor	Portal-only route
Microsoft MSSQL extension	SQL portal-versus-VS Code exercise	Portal-only SQL; SSMS alone does not demonstrate MSSQL UI
Azure Resources / Azure Functions extensions	Assigned VS Code discovery / local Function workflow	Portal-only route
.NET 10 SDK (not runtime only)	The supplied Function and local tracker projects	Portal SQL, Blob and Logs

C# tooling (C# extension; C# Dev Kit optional)	C# editor/debugger experience in VS Code	CLI dotnet build; portal-only
Functions Core Tools v4 compatible with .NET 10 isolated	Running HelloAzureFunction locally	Portal-only/cloud test-only Function demo
Azurite (VS Code extension recommended)	Local Function storage with UseDevelopmentStorage=true	Portal Blob exercise; local tracker
Azure CLI	Terminal Blob transfers and Day 1 remote KQL runner	Portal-only; local tracker checks
Git	Cloning/updating the repository	ZIP download; portal-only
Visual Studio with .NET 10 support	Optional alternative IDE; workloads appropriate to the assigned project	VS Code route; portal-only
SSMS	Optional alternative SQL management client	MSSQL extension route
SQL Server Express / LocalDB	Optional local SQL/AppServiceSqlSample path only	Azure SQL portal/MSSQL labs; local tracker
Node.js/npm	Only if using an approved npm installation path for tooling	Preinstalled/VS Code Azurite route

2. Azure sign-in, scope and permissions

Open https://portal.azure.com in a browser. Use the organiser-approved work account and complete MFA. In portal settings or directory/subscription controls, select the assigned tenant and subscription. Open the assigned resource group and one resource's Overview; record its non-secret name/type/region.

Seeing Overview proves management visibility, not SQL SELECT, Blob transfer, query access or permission to retrieve Function keys. The portal, Azure CLI, VS Code extensions and MSSQL connection can have separate sign-in contexts.

Ask the organiser for the assigned resource inventory, unique SQL suffix/private container and your allowed route. Do not create a replacement in your personal subscription when something is missing.

Exercise	Access the owner must prepare / verify
Discovery	Approved read visibility at the assigned resource scope
Azure SQL	Approved network path plus database authentication and SELECT; CREATE/ALTER/UPDATE only for an assigned disposable write route
Blob	Approved network path and scoped Blob data role; write route normally needs Storage Blob Data Contributor, not just management Reader
Function	Approved endpoint and private Function-key

	handling; publishing is separate explicit replacement authority
Log Analytics / Application Insights	Approved query rights and relevant table/resource scope; a known instrumented app for the live Day 2 route
Cost Management	Authorised cost visibility at a supported scope/agreement; reader access does not imply budget creation
Service Bus Explorer	Assigned isolated queue and approved Data Sender/Receiver access plus portal discovery and network path
Optional resource creation	Explicit creation authority at approved scope, policy, region, SKU, quotas and cost approval; no blanket IAM changes

3. Install tools using approved sources

Use company software distribution where required; ask IT before installations needing administrator rights. Choose the Windows architecture supported by your laptop and the tool. Restart VS Code and terminals after installation so PATH changes are loaded.

VS Code: https://code.visualstudio.com/download . In Extensions, verify the Microsoft publisher for Microsoft SQL Server (mssql), Azure Resources, Azure Functions and C# tooling. Install Azurite from its verified publisher if running the local Function.

.NET 10 SDK: https://dotnet.microsoft.com/download/dotnet/10.0 . Signing into C# Dev Kit or installing a runtime does not install an SDK. The supplied Function targets net10.0, Azure Functions v4 and dotnet-isolated; the other supplied applications also target net10.0.

Azure CLI: https://learn.microsoft.com/cli/azure/install-azure-cli-windows . Functions Core Tools: https://learn.microsoft.com/azure/azure-functions/functions-run-local . Use a current compatible v4 release; the major version alone is not proof of .NET 10 compatibility.

Git for Windows: https://git-scm.com/download/win . SSMS (optional): https://learn.microsoft.com/ssms/install/install . Visual Studio (optional) must support the .NET 10 SDK and the assigned workload; it is not required in addition to VS Code.

No fixed installer versions are prescribed beyond the project's supported runtime requirements. Do not upgrade the shared Function App or install tools on another person's machine to satisfy this list.

4. Download the whole attendee pack

Corporate training repository: https://github.com/alpaBuddhabhatti/LBS_Training2026. Choose Code > Download ZIP and extract the complete archive, or clone the repository. Open its root folder, which contains 00_START_HERE.md, Day 1, Day 2 and Labs. An organiser-supplied Attendee Pack - Core ZIP contains the same course materials. Keep each distribution together.

Git is optional. To clone the corporate training repository, run:

```
git clone https://github.com/alpaBuddhabhatti/LBS_Training2026.git
```

Without Git, select Code > Download ZIP in the repository and extract the archive. Open the extracted LBS_Training2026-main folder. For an organiser-supplied ZIP, open Attendee Pack - Core. Keep Day 1, Day 2 and Labs together for relative links.

Use a writable local folder outside Program Files. Replace placeholder paths with your actual extracted Attendee Pack - Core root. The VS Code terminal commands below use Windows PowerShell, not a Linux shell.

5. Validate only the tools your route needs

Run each line separately in a new PowerShell terminal. A missing command is a prerequisite blocker for that route, not a reason to continue as if the check passed. Do not copy the PowerShell prompt or expected output into commands.

```
$PSVersionTable.PSVersion
code --version
dotnet --list-sdks
dotnet --version
az version
func --version
git --version
```

Expected: PowerShell is available; installed tools report versions; dotnet --list-sdks includes a 10.0.x SDK; func reports a compatible v4 installation. Other SDKs can coexist. A different selected dotnet --version needs checking in the actual project folder; do not uninstall unrelated SDKs. code may be absent from PATH even when the editor opens; verify the installation and restart the terminal.

```
az login
az account list --query "[].{Name:name,Id:id,Tenant:tenantId}" -o table
az account set --subscription "YOUR-APPROVED-SUBSCRIPTION-ID"
az account show --query "{Name:name,Id:id,Tenant:tenantId}" -o table
```

Select only the assigned subscription after successful login. If the correct tenant is not selected, the organiser may direct you to az login --tenant YOUR-APPROVED-TENANT-ID. Do not switch accounts randomly or paste credentials into commands. CLI sign-in success does not prove service data rights; shared outputs must contain no tokens.

6. Confirm files, restore/build and local checks

From the extracted attendee-pack root, use the commands below. Build commands can restore dependencies from NuGet and create local bin/obj output. They do not deploy anything. Check each command succeeds before continuing; stop on an error and retain the non-secret message.

```
Set-Location "C:\YOUR-ATTENDEE-PACK"Get-LocationTest-Path
".\Labs\VsCodePublishBasics\HelloAzureFunction\HelloAzureFunction.csproj"Test-Path
".\Labs\LbsDocumentTracker\SampleData\cases.json"dotnet build
".\Labs\VsCodePublishBasics\HelloAzureFunction\HelloAzureFunction.csproj"dotnet build
".\Labs\LbsDocumentTracker\LbsDocumentTracker.csproj"dotnet run --project
```

```
".\Labs\LbsDocumentTracker\Checks\LbsDocumentTracker.Checks.csproj" --
"$PWD\Labs\LbsDocumentTracker\SampleData"
```

Expected: both Test-Path results True; successful builds; tracker checks print 20 PASS lines and exit 0. Immediately after a native command use $LASTEXITCODE to inspect its exit code. A partial PASS list followed by an exception is not success. The tracker checks require no Azure account or SQL instance.

7. Local Function smoke test - full developer route

Open the actual HelloAzureFunction project folder in VS Code. In terminal one, copy the example only when local.settings.json is absent. The example uses development storage and dotnet-isolated. Do not overwrite an existing settings file or show its private values on screen.

```
if (-not (Test-Path ".\local.settings.json")) {
    Copy-Item ".\local.settings.example.json" ".\local.settings.json"
}
```

Use Ctrl+Shift+P > Azurite: Start. Then start the host in terminal one. Wait for the ApplicationStatus GET route; keep this terminal running.

```
func start --port 7071
```

In terminal two, while terminal one remains running:

```
curl.exe --noproxy "*" --fail --max-time 20 http://127.0.0.1:7071/api/ApplicationStatus
```

Expected JSON: {"reference":"LBS-DEMO-1001","status":"Received","fictionalData":true,"message":"Hello from the VS Code publishing exercise."}

Return to terminal one, press Ctrl+C and wait for the prompt. Stop only your own Azurite instance when no longer needed. This proves a local response, not cloud deployment, cloud-key enforcement or SQL/Blob integration. No Azure publish is part of pre-course validation.

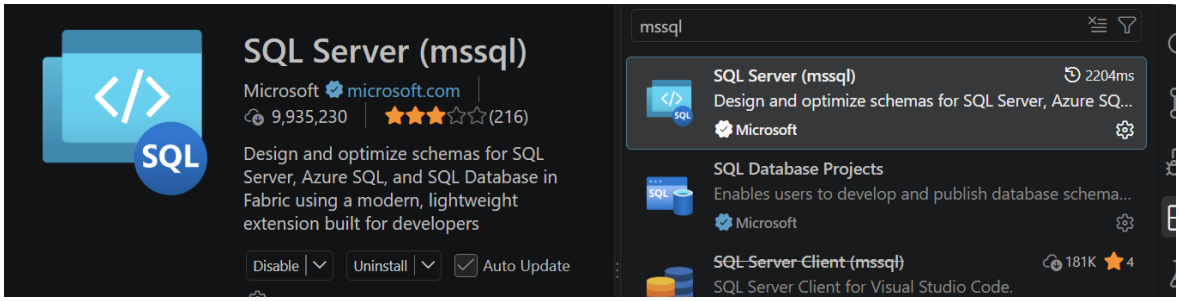
First-time SQL setup: VS Code first, then Microsoft SQL Server (mssql)

Required for the VS Code SQL route; optional for a portal-only participant. First install Visual Studio Code from https://code.visualstudio.com/download using the approved company process, then open it. Visual Studio Code is not the same product as Visual Studio.

1. In VS Code press Ctrl+Shift+X to open Extensions. Search mssql.

2. Choose SQL Server (mssql), published by Microsoft, extension ID ms-mssql.mssql. Click Install and reload if prompted. Do not select SQL Server Client (mssql) from another publisher. SQL Database Projects is not needed.

3. Disable and Uninstall buttons mean the extension is already installed and enabled in this context. A gear icon indicates an installed extension; verify it is enabled if commands are missing. The screenshot below shows the correct extension.



Correct Microsoft extension: Disable and Uninstall show it is already installed.

4. Open Labs\PortalAndVsCodeBasics\Samples\Sql\01-connection-check.sql from the attendee-pack root. Press Ctrl+Shift+P, search MS SQL: Connect, and select or create the assigned Azure SQL connection.

5. Enter the assigned Azure SQL hostname and database, use approved authentication, enable encryption and keep Trust server certificate = False for Azure. Keep passwords and sign-in codes private. Confirm the database is not master.

6. Choose Execute Query or press Ctrl+Shift+E to execute the connection-check query below. Expected: one row showing the assigned database, connected identity and server UTC time. A successful read does not prove CREATE/ALTER/UPDATE permission.

```
SELECT DB_NAME() AS DatabaseName,
       SUSER_SNAME() AS LoginName,
       SYSUTCDATETIME() AS CheckedAtUtc;
```

No database solution, SQL Database Project, local SQL Server Express or App Service sample is required for this Azure SQL connection. A .sql file supplies commands, not the connection. VS Code is local but the query executes against Azure SQL. Azure Resources sign-in alone is not an MSSQL connection.

If MS SQL: Connect is absent, confirm the exact Microsoft extension is enabled, open a .sql file, allow activation and reload if needed. If company policy blocks installation, ask IT and record Blocked; use an approved portal-only fallback rather than bypassing policy.

8. SQL tools - Azure core versus optional local SQL

For the core VS Code SQL exercise, install Microsoft SQL Server (mssql) and open a .sql file. Connect only to the assigned Azure hostname/database with approved authentication and network access. Use encryption and Trust server certificate = False for Azure.

Where your access is prepared, run SELECT DB_NAME() AS DatabaseName; as a read-only check. Expected: the assigned database, not master. No table creation or firewall change is needed for this check.

Portal-only SQL participants use the approved database Query editor and can omit MSSQL/SSMS. SSMS is an optional additional tool, not the database engine and not a substitute for demonstrating the VS Code route.

SQL Server Express or LocalDB is optional and only needed for a separately assigned local AppServiceSqlSample path. Do not install it for the Azure SQL, Function or local tracker exercises. The optional App Service/local SQL route is not included in this core pack.

9. Corporate network and proxy readiness

Ask IT to confirm the approved route to portal.azure.com, Microsoft sign-in/MFA endpoints, the assigned Azure service endpoints, NuGet package sources (including required CDN/package endpoints), github.com and required download hosts. A short hostname list is not a complete corporate allow-list; use the relevant product requirements and organisational policy.

HTTPS access to a homepage alone does not prove NuGet restore, Git clone or service data access works. The real restore/build and clone/download checks above exercise useful paths. Azure SQL may require its approved connection-policy ports or private-network VPN/DNS arrangement; ask IT, not a classroom-wide firewall bypass.

Configure proxies and enterprise certificates through approved processes. Do not disable TLS validation, set TrustServerCertificate for Azure, open all networks or put proxy passwords in scripts/screenshots. Test with the same VPN/network you will use at training.

If a proxy blocks the local request, the supplied curl loopback command bypasses a proxy only for that request. Do not globally disable a corporate proxy or endpoint protection.

Symptom	Safe recovery
Command not recognised	Confirm the approved installation, architecture and PATH; restart VS Code and terminal. Do not reinstall unrelated software blindly.
SDK not found / net10.0 unsupported	Install the .NET 10 SDK rather than runtime-only; check dotnet --list-sdks in the project context.
NuGet restore fails	Preserve the exact source/TLS/proxy error; confirm feed/network/certificate policy with IT. Do not disable verification.
Azure resource absent / denied	Check approved tenant, subscription filter and scope; contact organiser. No duplicate creation or Owner request.
Local Function storage error	Check Azurite is running and local-only settings are correct; never replace storage settings with a shared account key.
Port 7071 in use	Inspect ownership with the tutor or use an

	approved alternative consistently. Do not stop a process by name.
SQL timeout / login denied	Distinguish network route from authentication/data rights. Owner checks private routing or narrowly approved rules.
Cost or telemetry unavailable	Record live access Blocked and prepare the labelled Day 2 CSV/query fallback; do not invent a successful live check.

10. Day 1 readiness check - 09:45-10:00

0-3 minutes: welcome, chosen route and assigned inventory. Open the attendee pack and portal.

3-6 minutes: verify browser sign-in, correct tenant/subscription and one assigned resource. Do not create anything.

6-10 minutes: full-route attendees show tool versions and prior build/check results; portal-only attendees confirm their assigned read path. Do not start installing a missing SDK during this check.

10-13 minutes: verify the actual lab folder and required files; identify network or service-access blockers privately.

13-15 minutes: record Ready / Blocked / Not run by route and agree a fallback/owner. Continue the welcome and fundamentals at 10:00.

This uses the first 15 minutes of the existing 09:45-11:05 opening block; it does not extend the day. The separate 15-minute guided portal-navigation demonstration remains later in that opening block.

Readiness record	Your result / issue / owner
Chosen route and pack location	
Approved tenant/subscription/resource	
Tools required for my route	
Restore/build and tracker checks, if assigned	
Local Function smoke test, if assigned	
SQL/Blob/logs/cost access or fallback	
Corporate network/VPN/proxy	
Unresolved blocker and organiser/IT owner	

11. Finish and protect your environment

Keep real local settings, passwords, tokens and key-bearing URLs out of Git and shared evidence. Store only non-secret outcomes. Never paste an access token from az account get-access-token into notes.

Stop your own local hosts. Do not delete shared resources or remove existing role assignments. Do not commit generated bin/obj folders. This guide's checks make no claim of production readiness or live Day 2 telemetry.